

Architectures, Learning Loops, and Emergence in Agentic Services Computing

Sadaf Shafi
Institute of Computer Science
Georg-August-Universität Göttingen
Göttingen, Germany
sadaf.shafi@stud.uni-goettingen.de

Michael Bidollahkhani
Institute of Computer Science
Georg-August-Universität Göttingen
Göttingen, Germany
0000-0001-8122-4441
michael.bkhani@uni-goettingen.de

Julian M. Kunkel
Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen (GWDG)
Georg-August-Universität Göttingen
Göttingen, Germany
0000-0002-6915-1179
julian.kunkel@gwdg.de

Abstract—Large Language Model (LLM)-based agents are evolving from isolated task executors into interconnected societies of autonomous services capable of coordination, adaptation, and collective intelligence. This paper surveys and synthesizes recent advances in agentic services computing, LLM-based multi-agent systems, and language-augmented reinforcement learning to analyze how feedback-driven learning loops enable emergent behaviors at system scale. We organize the design space along four dimensions: perception and context modeling, autonomous decision-making, multi-agent collaboration, and evaluation with alignment and trustworthiness. Building on this analysis, we propose a reference architecture for feedback-driven LLM-agent societies that integrates reinforcement learning, verbal feedback, episodic memory, and governance layers. Finally, we identify open challenges, including cumulative learning without knowledge entropy, scalable coordination, and trustworthy evolution, outlining a research agenda for engineering reliable emergent agentic systems.

Index Terms—Agentic Services Computing, Large Language Models, Multi-Agent Systems, Emergent Behavior, Reinforcement Learning, Feedback Loops, Autonomous Agents, AI Architectures

I. INTRODUCTION

Large Language Models (LLMs) have enabled a new generation of autonomous agents that move beyond static input-output behavior toward persistent, goal-directed, and interactive systems. Increasingly, such agents are deployed not in isolation, but as collections of interacting services that collectively solve complex tasks. This paradigm, referred to as Agentic Services Computing (ASC) [1], emphasizes societies of LLM-powered agents capable of perception, reasoning, collaboration, and continuous learning.

Despite rapid progress, the field lacks a unified systems-level view that connects (i) architectural building blocks, [2] (ii) learning and feedback loops [3], and (iii) the emergence of collective behaviors [4]. Existing work often focuses either on agent-level capabilities (tool use, planning, memory) or

on multi-agent coordination strategies, while the mechanisms that transform local adaptation into reliable system-scale emergence remain under-specified. In particular, feedback-driven learning loops raise both opportunities and risks: they can improve coordination and robustness, but may also amplify instability, drift, or misalignment if not governed.

This paper addresses this gap by synthesizing recent advances across agentic services computing, LLM-based multi-agent systems, and language-integrated reinforcement learning. We organize the design space along four dimensions: perception and context modeling, autonomous decision-making, multi-agent collaboration, and evaluation with alignment and trustworthiness. Building on this synthesis, we propose a reference architecture for feedback-driven LLM-agent societies that integrates reinforcement learning, verbal feedback, episodic memory, and governance layers. Finally, we outline open challenges and a research agenda for engineering reliable emergent agentic systems.

A. Contributions

Our contributions are:

- A structured synthesis of recent literature connecting ASC, LLM-based multi-agent systems, and language-augmented reinforcement learning.
- A four-dimensional design space for analyzing and comparing agentic service architectures.
- A reference architecture that explicitly models feedback-driven learning loops, memory, coordination, and governance in emergent agent societies.
- A set of open challenges and research directions, including cumulative learning without knowledge entropy, scalable coordination, and trustworthy evolution.

II. RELATED WORK

Research on LLM-based agents and multi-agent systems has expanded rapidly, spanning surveys, conceptual taxonomies, and system frameworks. This section groups prior work into four categories: LLM-based autonomous agents, agentic services computing, LLM-augmented multi-agent systems, and language-integrated reinforcement learning.

A. LLM-Based Autonomous Agents

Several surveys characterize the emergence of autonomous agents built around LLMs, highlighting capabilities such as tool use, memory, planning, and self-reflection. These systems typically treat the LLM as a cognitive core responsible for reasoning and instruction following, augmented with retrieval mechanisms and external tools.

Conceptual distinctions between *AI Agents* and *Agentic AI* emphasize that while individual agents execute bounded tasks, agentic AI denotes system-level intelligence arising from coordinated collections of agents. [2], [5], [6]

B. Agentic Services Computing

Agentic Services Computing (ASC) frames agents as persistent, autonomous services that interact dynamically across their lifecycle. Rather than static service orchestration, ASC emphasizes self-organization, role negotiation, and adaptive service composition, enabling complex workflows to emerge from decentralized interactions.[1], [3]

C. LLM-Based Multi-Agent Systems

Recent work demonstrates that societies of LLM-powered agents can exhibit collective intelligence, role specialization, and emergent communication. These systems range from social simulation platforms to swarm-inspired environments, showing that global behaviors can arise from localized decision-making. [4], [7], [8], [9], [10], [11]

D. Language-Augmented Reinforcement Learning

Hybrid frameworks combine LLMs with reinforcement learning by using language to generate subgoals, coordinate agents, or shape rewards. These approaches improve coordination and generalization in multi-agent reinforcement learning environments. [12]

III. FOUR-DIMENSIONAL DESIGN SPACE

We organize the design of agentic services along four orthogonal dimensions.

Figure 1 summarizes how the literature surveyed in this work is grouped into three high-level research archetypes—Service-Oriented (ASC, Workflow), Social/Swarm (M-LLM, Swarm), and Learning-Centric (Self-Evolving, MARL)—and positioned along the four design dimensions shown in the radar plot.

A. Perception and Context Modeling

Agents must transform raw observations into structured representations. Multimodal encoders and memory retrieval mechanisms allow agents to maintain situational awareness across time.

B. Autonomous Decision-Making

Decision-making combines LLM-based reasoning with policy networks or rule-based components, enabling both symbolic reasoning and statistical learning.

C. Multi-Agent Collaboration

Agents exchange messages, negotiate roles, and decompose tasks into subgoals. Communication protocols may be emergent or predefined.

D. Evaluation, Alignment, and Trustworthiness

Safety filters, audit trails, and governance policies ensure agents operate within acceptable bounds.

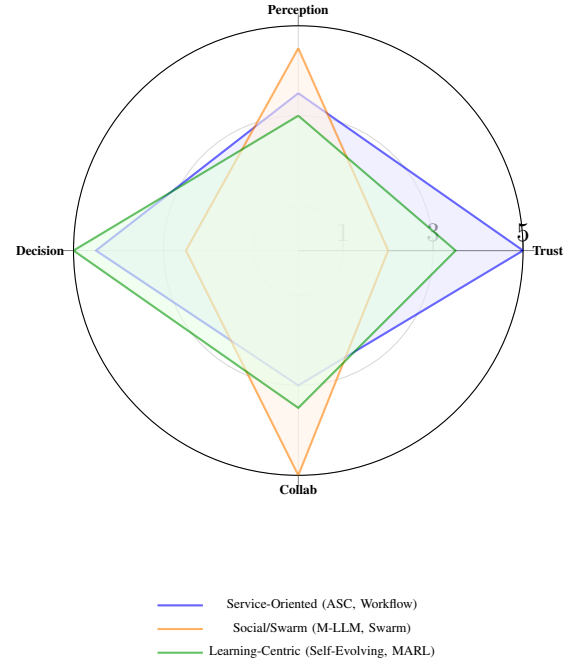


Fig. 1. Comparative analysis of research archetypes. No single framework achieves maximum coverage, highlighting the inherent trade-offs between coordination flexibility and governance rigor.

IV. REFERENCE ARCHITECTURE FOR FEEDBACK-DRIVEN AGENTIC SERVICES

This section proposes a reference architecture for feedback-driven Agentic Services Computing (ASC). The goal is not to prescribe a single implementation, but to define a modular systems blueprint that can be used to (i) compare architectures across the literature, (ii) guide implementations, and (iii) identify where learning loops and governance controls should be placed to support reliable emergence.

A. Architecture Overview

We model an agentic service society as a set of interacting agents $\{a_1, a_2, \dots, a_N\}$ operating in an environment with (possibly partial) observability. Each agent instantiates local capabilities (perception, reasoning, memory, action execution), while selected components may be shared or coordinated

TABLE I
COMPARISON OF SURVEYED LLM-BASED MULTI-AGENT FRAMEWORKS AND SURVEYS

Paper	Primary Domain	Agent Design Core	Feedback Mechanism	Key Contribution
ASC [1]	Services Computing	Lifecycle-aware Services	Verbal RL / Natural Language	Unified governance for self-improving service societies.
Taxonomy [6]	Conceptual Framework	Modular AI Agents	Causal Modeling / ReAct	Paradigm shift from task automation to coordinated autonomy.
Social Paradigm [8]	Social Simulation	6-Tier Complexity Continuum	Social Observation	Mapping development from static tools to adaptive systems.
Swarm Intel [10]	Classic MAS / Swarms	NetLogo Integration	Iterative Prompt Tuning	Hybrid groups (LLM + Rule-based) performance gains.
MARL-LLM [12]	Game Environments	LLM-Augmented MARL	Hybrid Reward + Subgoal Loss	Symbolic reasoning for better generalization in MARL.
Socialized Learning [7]	Robotics / Assembly	Multimodal (Text + Vision)	Observational Social Learning	Machine social cognition via shared multimodal context.
ABMS Survey [9]	Modeling & Sim	Heterogeneous Agents	Short/Long-term Memory	Categorization of emergence across Physical and Cyber domains.
Autonomous Agents [5]	LLM Agents	4-Module Cognitive core	Env / Model Self-Correction	Systematic taxonomy of "Mechanism Engineering" for agents.
Self-Evolving [3]	Lifelong Systems	Modular "Search Space"	Training/Test-time Feedback	The MASE paradigm and the Three Laws of Agent Evolution.
Collab Mech [11]	Collective Intel	"Digital Species" Roles	Debate / Competitive Rivalry	Taxonomy of Cooperation, Competition, and Coopetition.
Workflow/Infra [2]	System Lifecycle	5-Component Lifecycle	Multi-source (Human/Peer)	Blueprint for scaling; addressing Group-think vs. Intelligence.
Progress Survey [4]	General AI Progress	Profiles (Data vs. Model)	Capability Acquisition	Mapping Problem Solving vs. World Simulation benchmarks.

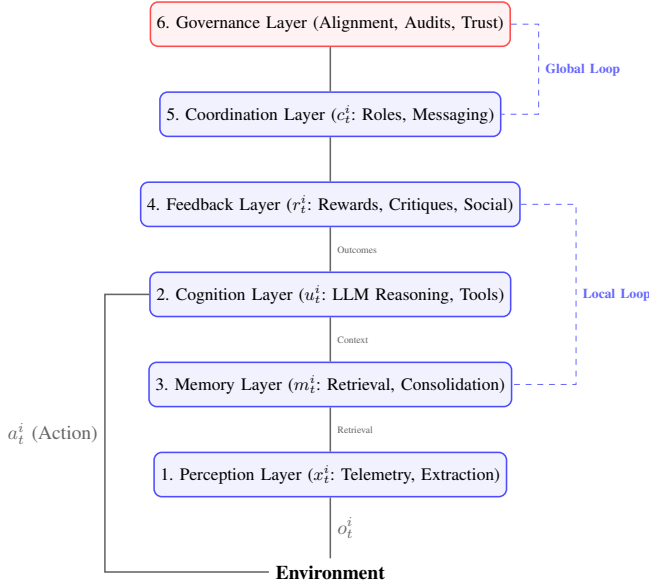


Fig. 2. Reference Architecture for Feedback-Driven Agentic Services Computing (ASC).

globally (e.g., governance policies, shared memory indices, or centralized critics).

We organize the reference architecture into six layers:

- 1) **Perception Layer:** transforms raw observations (text, structured telemetry, multimodal inputs) into representations usable by downstream components.
- 2) **Cognition Layer:** produces decisions through LLM

reasoning, tool invocation, and optional policy networks.

- 3) **Memory Layer:** maintains short-term context and long-term episodic/semantic memory with retrieval and consolidation.
- 4) **Feedback Layer:** converts outcomes into learning signals, including scalar rewards, verbal critiques, and social feedback.
- 5) **Coordination Layer:** enables multi-agent collaboration through messaging, role allocation, and subgoal decomposition.
- 6) **Governance Layer:** enforces alignment, auditability, and trust constraints across the society.

B. Layer Interfaces and Data Flows

The architecture is characterized by explicit interfaces between layers. Let o_t^i denote the observation received by agent a_i at time t .

- **Perception:** $x_t^i = f_{\text{perc}}(o_t^i)$, where x_t^i is a structured representation (e.g., state summary, extracted entities, embeddings).
- **Memory Retrieval:** $m_t^i = f_{\text{mem}}(x_t^i)$, returning relevant episodic traces, skills, or documents.
- **Decision:** $u_t^i = f_{\text{cog}}(x_t^i, m_t^i, c_t^i)$, where c_t^i includes coordination signals such as assigned roles or received messages.
- **Action Execution:** agent outputs actions a_t^i (tool calls, messages, environment actions) derived from u_t^i .
- **Feedback:** learning signal $r_t^i = f_{\text{fb}}(o_{t+1}^i, a_t^i, \Delta_t^i)$ where Δ_t^i summarizes outcome changes such as task success,

latency, cost, safety violations, or human ratings.

This formulation allows agents to be implemented as purely LLM-driven planners, purely RL-driven policies, or hybrid systems where LLMs generate subgoals and RL optimizes execution policies.

Algorithm 1: ASC Local Agent Execution and Self-Correction

Input: Raw observation o_t^i , Coordination context c_t^i , Local Memory $\mathcal{M}^i$

Output: Action a_t^i , Learning signal r_t^i

// 1. *SENSE: Perception and Telemetry Extraction*

1. $x_t^i \leftarrow f_{perc}(o_t^i)$ // Extract state summary and SLA metrics

// 2. *RETRIEVE: Contextual Memory Access*

2. $q_t^i \leftarrow \text{embed}(x_t^i \cup c_t^i)$ // Generate query vector

3. $m_t^i \leftarrow \text{TopK}(\mathcal{M}^i, q_t^i, k = 5)$ // Retrieve episodic traces/skills

// 3. *DECIDE: Cognitive Reasoning and Tool Selection*

4. $P_t^i \leftarrow \text{ComposePrompt}(x_t^i, m_t^i, c_t^i)$ // Build system/user prompt

5. $u_t^i, \tau_t^i \leftarrow \text{LLM}(\text{Prompt} = P_t^i)$ // Decision u and reasoning trace τ

// 4. *ACT: Service Execution*

6. $a_t^i \leftarrow \text{map}(u_t^i, \text{ToolSet } \mathcal{T})$ // Dispatch to API or Environment

// 5. *FEEDBACK: Outcome Evaluation*

7. $r_t^i, \gamma_t^i \leftarrow \text{Evaluate}(o_{t+1}^i, a_t^i, \text{SLA_Policy } \mathcal{S})$ // Signal r and critique γ

// 6. *UPDATE: Local Knowledge Consolidation*

8. **if** $\text{Quality}(r_t^i) < \text{Threshold}$ **or** $r_t^i = \text{Success}$ **then**

9. $e_{new} \leftarrow \text{SynthesizeExperience}(\tau_t^i, r_t^i, \gamma_t^i)$

10. $\mathcal{M}_{t+1}^i \leftarrow \text{Consolidate}(\mathcal{M}^i, e_{new})$ // Update local vector index

11. **end if**

Fig. 3. Detailed execution flow for an ASC agent, emphasizing the transformation of outcomes into consolidated episodic memory.

C. Local and Global Learning Loops

Feedback-driven ASC requires clarifying where learning occurs:

- **Local loop:** each agent updates policies, prompts, memories, or skills from its own feedback stream.
- **Global loop:** the society aggregates traces and feedback to improve coordination policies, shared tools, governance rules, or shared memory indices.

Local learning improves individual competence, while global learning is responsible for system-level emergence properties such as division of labor, stable communication protocols, and coordinated planning.

D. Governance and Trustworthiness Controls

Because feedback loops can amplify errors, the governance layer must support:

- **Policy constraints:** hard rules and soft constraints on allowable actions.
- **Audit logs:** complete traceability of decisions, prompts, tool calls, and feedback.
- **Safety checks:** detection and mitigation of unsafe behaviors and reward hacking.
- **Evaluation gates:** staged deployment where learned changes must pass validation tests.

These controls separate exploratory learning from production deployment and are critical for trustworthy evolution.

V. FEEDBACK-DRIVEN LEARNING LOOPS

Feedback transforms static agents into evolving entities.

A. Reinforcement Learning

Agents receive scalar rewards from the environment and optimize policies using algorithms such as PPO.

B. Verbal Feedback

Natural-language critiques generated by humans or agents serve as dense reward signals.

C. Social Feedback

Agents learn by observing peers and imitating successful behaviors.

D. Human-in-the-Loop Feedback

Human oversight corrects errors and steers learning.

E. Collective-System Evaluation Parameters

Feedback-driven agentic service societies can be viewed as collective adaptive systems composed of interacting autonomous components. Such systems are commonly characterized using population-level parameters that capture organization, coordination, adaptability, and stability. We adopt this perspective and define a set of collective-system evaluation parameters that are observable from execution traces and directly relevant to agentic functionality.

These parameters support both evaluation and optimization of feedback-driven learning loops.

Parameter Descriptions. *CD* denotes the average number of meaningful inter-agent interactions per decision step and reflects coordination intensity within the society. *GAR* measures the fraction of steps in which agent-level subgoals remain consistent with the global task objective. *RSpI* is computed as the entropy of action-type distributions per agent; lower values indicate stronger functional specialization. *RR* represents the probability that the society returns to a valid execution trajectory after a failure or critique event. *SD* captures the diversity of solution strategies observed across runs or iterations. *CT* measures the number of successful task completions per unit time or per fixed compute budget. *BV* corresponds to the variance of action distributions over

TABLE II
COMPONENT PLACEMENT IN ASC SOCIETIES: LOCAL VS. GLOBAL ARCHITECTURES

Layer	Local (Agent-Centric)	Global (Society-Centric)	Key Reference(s)
Perception	Individual telemetry processing and local state extraction.	Shared environment state and global world-model consistency.	[7]
Cognition	LLM reasoning, local planning, and private tool invocation.	Shared prompt templates and centralized policy guidelines.	[5]
Memory	Private episodic traces and short-term conversation context.	Shared vector indices and common knowledge bases.	[2]
Feedback	Internal self-critique and verbal reinforcement loops.	Collective reward signals and peer-evaluation datasets.	[1], [12]
Coordination	Peer-to-peer messaging and local subgoal adherence.	Global role registry and centralized task decomposition.	[4], [11]
Governance	Local safety guardrails and input/output filtering.	Global audit logs, alignment policies, and trust metrics.	[1], [6]

Algorithm 2: Global Governance and Collective Evolution Loop

Input: Collective traces $\mathcal{E} = \{\tau^1, \dots, \tau^N\}$, Global Policy Ω_t , Shared Knowledge Θ_t

Output: Optimized Policy Ω_{t+1} , Refined Shared Index Θ_{t+1}

// 1. AUDIT: Automated Governance and Constraint Checking

```

1.  $\mathcal{V}, \mathcal{L} \leftarrow \emptyset$ 
2. foreach  $\tau^i \in \mathcal{E}$  do
3.   if Violates( $\tau^i$ , Safety_Policy  $\mathcal{S}$ ) = True then
4.      $\mathcal{V} \leftarrow \mathcal{V} \cup \{\tau^i\}$  // Mark for exclusion from learning
5.      $\mathcal{L} \leftarrow \mathcal{L} \cup \text{AuditLog}(\tau^i)$  // Record violation for audit
6.   end if
7. end
```

// 2. CONSOLIDATE: Cross-Agent Knowledge Distillation

```

8.  $\mathcal{E}_{\text{valid}} \leftarrow \mathcal{E} \setminus \mathcal{V}$ 
9.  $\mathcal{K}_{\text{new}} \leftarrow \text{LLM\_Summarize}(\text{SuccessfulPatterns}(\mathcal{E}_{\text{valid}}))$ 
10.  $\Theta_{t+1} \leftarrow \text{Merge}(\Theta_t, \mathcal{K}_{\text{new}}, \text{strategy}=\text{NoEntropy})$  // Prevent redundancy
```

// 3. EVOLVE: Meta-Reasoning and Topology Adjustment

```

11.  $\Delta_{\text{perf}} \leftarrow \text{Compare}(\text{BatchMetrics}(\mathcal{E}), \text{Target\_SLA})$ 
12. if  $\Delta_{\text{perf}} < 0$  then
13.    $\Omega_{t+1} \leftarrow \text{RefineMetaPrompt}(\Omega_t, \text{Failures}(\mathcal{V}))$  // Update system instructions
14.   Broadcast( $\Omega_{t+1}$ ) // Deploy new rules to all agents
15. else
16.    $\Omega_{t+1} \leftarrow \Omega_t$ 
17. end if
```

Fig. 4. The Global Evolutionary Loop, formalizing the transition from individual agent traces to updated society-level policies and knowledge.

TABLE III
TAXONOMY OF FEEDBACK MECHANISMS IN AGENTIC SERVICES COMPUTING

Category	Source	Modality	Key Reference(s)
Reinforcement	Environment	Scalar Reward	[12], [3]
Verbal Critique	Self / Peer	Natural Language	[1], [9]
Social Signal	Peer Agents	Observational	[7], [8]
Human-in-Loop	Human User	Preference / Text	[5], [2]

76
TABLE IV
COLLECTIVE-SYSTEM EVALUATION PARAMETERS FOR FEEDBACK-DRIVEN AGENTIC SERVICE SOCIETIES.

Collective Property	Parameter	Symbol
Coordination Intensity	Coordination Density	CD
Goal Consistency	Goal Agreement Rate	GAR
Functional Differentiation	Role Specialization Index	$RSpI$
Adaptation	Recovery Rate	RR
Strategy Diversity	Strategy Diversity	SD
Collective Efficiency	Collective Throughput	CT
Stability	Behavioral Variance	BV
Robustness	Failure Tolerance	FT

time, where lower variance indicates behavioral convergence. FT denotes the fraction of runs that succeed despite blocked actions or partial agent failures.

VI. CASE STUDIES OF EMERGENT COORDINATION AND COMMUNICATION

This chapter summarizes two recent case studies that illustrate how multi-agent learning systems can exhibit qualitatively new collective behaviors—both beneficial and brittle—when optimizing for task reward under interaction constraints. We use these examples to ground the paper’s discussion of coordination, feedback, governance, and trace-derived evaluation parameters.

A. Case Study: Emergent “Charging” Strategy via Reinforcement Learning

Context. In the Google Research Football (GRF) 11×11 full-game scenario, independent agents are trained with Independent Proximal Policy Optimization (IPPO) against a built-in “hard” AI opponent (difficulty 1.0). [13]

Emergent behavior (coordinated planning). Rather than converging to conventional football tactics (e.g., formation maintenance and balanced offense/defense), the learned team develops a collective “charging forward” strategy: agents abandon defensive positioning and rush into the opponent half. The behavior is coordinated in the sense that its effectiveness relies on many agents simultaneously flooding the offensive region. The authors attribute the emergence to an exploitable weakness in the built-in AI’s logic (involving offside-related dynamics), which becomes advantageous only under synchronized team movement.

Quantitative results.

- **Performance:** the emergent coordinated strategy achieves an average win rate exceeding 90% against the hard-coded baseline opponent.
- **Efficiency:** in high-resource settings, agents converge rapidly, surpassing the baseline in approximately 4.9 hours of training.
- **Dominance vs. instability:** despite strong performance against the fixed baseline, the strategy corresponds to a local optimum and can be fragile when evaluated against a broader population of policies (e.g., in Population-Based Training), highlighting overfitting to opponent-specific dynamics.

B. Case Study: Emergent Communication Efficiency in Information-Asymmetric Tasks

Context. In many multi-agent LLM systems, agents inherit human-alignment norms that encourage verbose, polite exchanges (e.g., repeated context, hedging, and pleasantries). The OPTIMA framework studies whether a multi-agent system can instead learn a communication norm that is optimized for task-level effectiveness and efficiency in information-asymmetric question answering (e.g., multi-hop settings such as 2WikiMultiHopQA). [14]

Emergent behavior (communication norm shift). Under OPTIMA, a society of Llama 3 agents develops a compressed, information-dense interaction style—a “dialect” that discards conventional conversational etiquette in favor of concise, high-signal messages. [14]

Quantitative results.

- **Norm shift (efficiency):** communication volume is reduced by more than 90%, using less than 10% of the tokens compared to baseline multi-agent interaction patterns.
- **Performance spike (effectiveness):** despite shorter messages, the optimized system achieves a reported $2.8 \times$ performance gain relative to vanilla multi-agent baselines.

- **Scaling behavior:** improved token efficiency enables better inference-time scaling by permitting more voting/sampling within a fixed compute budget, increasing robustness of decisions.

Both case studies suggest that (i) coordination and communication can shift abruptly as agents discover new reward-relevant interaction patterns, and (ii) strong task reward can coincide with brittle generalization. In our framework, such phenomena motivate jointly tracking outcome metrics (e.g., success rate) with coordination and governance signals (e.g., coordination density, specialization, and policy-violation rates), and evaluating across diverse opponent/task distributions to detect overfitting to narrow interaction dynamics.

VII. EMERGENT BEHAVIORS

Emergent behaviors arise when collective dynamics exceed the capabilities of any single agent.

Observed phenomena include:

- Role specialization
- Coordinated planning
- Norm formation
- Self-healing

A. Observed Emergence Signals in the Prototype

While the prototype operates at small scale, it still enables observation of early emergence signals that can be quantified from logs. We emphasize that these signals are not claims of general emergent intelligence, but measurable indicators of organization and coordination dynamics.

VIII. CHALLENGES AND OPEN PROBLEMS

A. Knowledge Entropy

A persistent challenge in agentic service societies is *knowledge entropy*: as experiences accumulate, memories can become redundant, inconsistent, or overly state-dependent, leading to degraded retrieval quality and “personality drift” across tasks. [2]

Mitigation directions. Inspired by self-evolving agent designs, we emphasize semantic consolidation and selective retention. [2]

- *Semantic memory consolidation:* add an “evolution” module that periodically self-reflects on past trajectories to refine policies, compress lessons learned, and resolve contradictions.
- *Short-term vs. long-term separation:* use Retrieval-Augmented Generation (RAG) to keep transient context in a session buffer while maintaining a curated long-term store for stable knowledge.
- *Memory pruning via retrieval signals:* deploy a memory retriever that detects near-duplicates and prunes low-value episodic traces, retaining only consistent, high-utility “lessons learned” to reduce drift.

TABLE V
EMERGENCE SIGNALS AND HOW THEY ARE OPERATIONALIZED IN THE
PROTOTYPE.

Phenomenon	Operational Signal	Description and Measurement Guidance
Role specialization	<i>RSpI</i> (role entropy)	Agents repeatedly handle distinct responsibilities (planning, retrieval, execution, critique); quantify via entropy of action types per role.
Coordinated planning	<i>DD</i> and subgoal reuse	Planner produces nested subgoals; measure maximum depth and whether subgoals are reused across attempts.
Norm formation	Constraint adherence trend	Repeated critiques lead to stable formatting and policy compliance; quantify via decreasing <i>PVR</i> and fewer corrections.
Self-healing behavior	Recovery after critique	After failure, society converges to a compliant output within fewer steps; quantify via reduction in <i>CS</i> following critique.

B. Scalability

Scalability is constrained by coordination overhead: naive fully connected communication patterns and verbose messages can drive quadratic growth in tokens, latency, and cost as the number of agents increases. [14]

Mitigation directions. We highlight adaptive topology control and communication pruning.

- *Adaptive topology*: move from rigid, fully connected interaction graphs to self-evolving topologies that rewire communication links based on task structure and information needs.
- *Communication efficiency (OPTIMA)*: adopt efficiency-oriented protocols (e.g., OPTIMA) that optimize interaction norms, reporting large gains in effectiveness while reducing token use to a small fraction of baseline. [14]
- *Hierarchies and handoffs*: use hierarchical role allocation and scoped “handoffs” so only necessary peers exchange

information, reducing broadcast and redundant context sharing.

C. Evaluation of Emergence

A core methodological gap is the lack of standardized, multi-level evaluation for emergent behavior: systems may display coordination, norm formation, or “wisdom of crowds” effects that are hard to compare across tasks and implementations. [8]

Mitigation directions. We recommend multilevel frameworks and mechanism engineering.

- *Multilevel categorization*: adopt a structured continuum (e.g., Level 0–Level 5) to classify systems by autonomy and social complexity, enabling more meaningful cross-paper comparisons. [8]
- *OODA-lens instrumentation*: use the Observe–Orient–Decide–Act loop to instrument agent societies and track phenomena such as opinion dynamics, coordination phases, and norm shifts.
- *Mechanism engineering*: evaluate whether iterative consensus-building actually improves decision quality (“wisdom of crowds”) by testing controlled interventions (e.g., voting, critique rounds, diversity prompts) and measuring robustness.

D. Trust and Governance

Trustworthy operation requires that autonomous evolution remains auditable and constrained: without explicit governance, agents may exhibit reward hacking, policy violations, or unsafe tool use that is difficult to attribute post hoc. [1], [2], [5], [11]

Mitigation directions. We propose unified governance and evaluation gates.

- *Unified governance model*: apply both hard and soft constraints to agent behaviors, treating heterogeneous agent roles as distinct “digital species” that require oversight. [1], [11]
- *Audit logs by default*: maintain immutable audit logs covering reasoning trajectories, tool calls, and feedback signals to enable traceability and incident analysis. [1]
- *Evaluation gates*: separate exploratory learning from production deployment via gated rollouts and regression checks, mitigating risks such as reward hacking or safety violations. [2], [5]

E. Deployment and Scalability Considerations

From a services computing perspective, deployment choices shape both coordination cost and governance feasibility. At scale, naive all-to-all messaging induces quadratic overhead, and memory growth can degrade retrieval quality.

IX. CONCLUSION

This paper presented a systems-level synthesis of agentic services computing, focusing on architectures, learning loops, and emergent behaviors. We proposed a reference architecture

TABLE VI
DEPLOYMENT AND SCALABILITY CONSIDERATIONS FOR AGENTIC
SERVICE SOCIETIES.

Aspect	Issue	Mitigation Strategy (System-Oriented)
Communication	Message explosion with agent count	Use role hierarchies, sparse routing, and brokered coordination instead of all-to-all chat.
Latency	Multi-step tool orchestration	Batch tool calls, cache intermediate results, and prioritize critical-path actions in the planner.
Memory growth	Episodic store becomes redundant/inconsistent	Periodic consolidation, deduplication, aging policies, and task-scoped memory partitions.
Governance	Hard to audit evolving prompts and policies	Enforce gated deployments, immutable audit logs, and regression checks before enabling learned updates.

and identified open challenges that must be addressed to engineer reliable LLM-agent societies.

A central takeaway is that rigorous evaluation remains a bottleneck: emergence is often only apparent post hoc, and visualizing or attributing performance differences *before* versus *after* an emergent shift is difficult to implement reliably in real systems.

We also find that effective agents require feedback signals, but the appropriate form and granularity of feedback likely depends on the agent type (e.g., planner vs. executor vs. critic) and the operational context; systematically designing such role-conditioned feedback is a key direction for future work.

Future work will focus on building prototypes and benchmarks to empirically evaluate feedback-driven emergence.

ACRONYMS

AI	Artificial Intelligence
LLM	Large Language Model
MAS	Multi-Agent System
ASC	Agentic Services Computing
RL	Reinforcement Learning
MARL	Multi-Agent Reinforcement Learning
RAG	Retrieval-Augmented Generation
PPO	Proximal Policy Optimization
OODA	Observe-Orient-Decide-Act

REFERENCES

- [1] S. Deng et al., “Agentic services computing,” *arXiv preprint arXiv:2509.24380*, 2025.
- [2] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, “A survey on llm-based multi-agent systems: Workflow, infrastructure, and challenges,” *Vicinagearth*, vol. 1, no. 1, p. 9, 2024.

- [3] J. Fang et al., “A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems,” *arXiv preprint arXiv:2508.07407*, 2025.
- [4] T. Guo et al., “Large language model based multi-agents: A survey of progress and challenges,” in *IJCAI*, 2024.
- [5] L. Wang et al., “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186 345, 2024.
- [6] R. Sapkota, K. I. Roumeliotis, and M. Karkee, “Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges. 2025,” *Accessed: Aug*, vol. 13, 2025.
- [7] S. Akin, S. T. Tiwari, R. Bhattacharya, S. A. Raman, K. Mohanty, and S. Krishnan, “Socialized learning and emergent behaviors in multi-agent systems based on multimodal large language models,” *arXiv preprint arXiv:2510.18515*, 2025.
- [8] J. Haase and S. Pokutta, “Beyond static responses: Multi-agent llm systems as a new paradigm for social science research,” *arXiv preprint arXiv:2506.01839*, 2025.
- [9] C. Gao et al., “Large language models empowered agent-based modeling and simulation: A survey and perspectives,” *Humanities and Social Sciences Communications*, vol. 11, no. 1, pp. 1–24, 2024.
- [10] C. Jimenez-Romero, A. Yegenoglu, and C. Blum, “Multi-agent systems powered by large language models: Applications in swarm intelligence,” *Frontiers in Artificial Intelligence*, vol. 8, p. 1 593 017, 2025.
- [11] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O’Sullivan, and H. D. Nguyen, “Multi-agent collaboration mechanisms: A survey of llms, 2025,” *URL https://arxiv.org/abs/2501.06322*,
- [12] Z. Li, S. Campos, and N. Wang, *Language-driven coordination and learning in multi-agent simulation environments*, 2025. arXiv: 2506 . 04251 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/2506.04251>
- [13] Y. Song et al., “An empirical study on google research football multi-agent scenarios,” *Machine Intelligence Research*, vol. 21, no. 3, pp. 549–570, Jan. 2024, ISSN: 2731-5398. DOI: 10.1007/s11633-023-1426-8 [Online]. Available: <http://dx.doi.org/10.1007/s11633-023-1426-8>
- [14] W. Chen, J. Yuan, C. Qian, C. Yang, Z. Liu, and M. Sun, *Optima: Optimizing effectiveness and efficiency for llm-based multi-agent system*, 2025. arXiv: 2410.08115 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2410.08115>